

CO4 – ASSESSMENT TOOL 1: CODE CHALLENGE

Database Management Systems (CSA05) — MySQL Implementation

CO4: File Organization, Indexing & Hashing Techniques

All answers below are written as executable MySQL (8.0+) scripts — tables, stored procedures/functions, and test CALL/SELECT statements — instead of Python/C, per your request. Run the Preliminary Setup once first. Each question has a code block followed by a bordered box: paste your MySQL Workbench / terminal output there before submission.

PRELIMINARY SETUP – Database & Table Creation

Run this once before attempting any question below. It creates the working database and the base 'students' table used across several questions.

```
-- Create and select the working database
CREATE DATABASE IF NOT EXISTS dbms_lab;
USE dbms_lab;

-- Base table used by several questions
CREATE TABLE students (
    regno      INT PRIMARY KEY,
    name       VARCHAR(50),
    department VARCHAR(30),
    marks      INT
);

INSERT INTO students (regno, name, department, marks) VALUES
(101, 'Arun', 'CSE', 85),
(102, 'Bala', 'ECE', 72),
(103, 'Chitra', 'IT', 91),
(104, 'Deepa', 'CSE', 65),
(105, 'Elango', 'MECH', 78),
(106, 'Farah', 'CSE', 88),
(107, 'Gopal', 'ECE', 55),
(108, 'Hema', 'IT', 95),
(109, 'Iniya', 'CSE', 60),
(110, 'Jagan', 'MECH', 73);

SELECT * FROM students;
```

Question 1

Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

Note: MySQL version: a heap file has no physical ordering, so records are stored in insertion (slot) order and deletion is logical (status flag), matching real heap-file behaviour where slots are not physically shifted.

```
USE dbms_lab;

CREATE TABLE heap_file (
    slot_no INT AUTO_INCREMENT PRIMARY KEY,    -- physical insertion order
    regno   INT,
    name    VARCHAR(50),
    marks   INT,
    status  VARCHAR(10) DEFAULT 'ACTIVE'
);

DELIMITER //

CREATE PROCEDURE heap_insert(IN p_regno INT, IN p_name VARCHAR(50), IN p_marks INT)
BEGIN
    INSERT INTO heap_file(regno, name, marks, status)
    VALUES (p_regno, p_name, p_marks, 'ACTIVE');
END //

CREATE PROCEDURE heap_delete(IN p_regno INT)
BEGIN
    -- logical delete: heap files keep the slot but mark it deleted
    UPDATE heap_file SET status = 'DELETED' WHERE regno = p_regno;
END //

CREATE PROCEDURE heap_scan()
BEGIN
    -- sequential scan reads every slot in physical insertion order
    SELECT slot_no, regno, name, marks, status
    FROM heap_file
    ORDER BY slot_no;
END //

DELIMITER ;

-- Test
CALL heap_insert(201, 'Kavin', 80);
CALL heap_insert(202, 'Lalitha', 70);
CALL heap_insert(203, 'Manoj', 90);
CALL heap_delete(202);
CALL heap_scan();
```

OUTPUT

```

mysql>
mysql> DELIMITER ;
mysql>
mysql> -- Test
mysql> CALL heap_insert(201, 'Kavin', 80);
Query OK, 1 row affected (0.01 sec)

mysql> CALL heap_insert(202, 'Lalitha', 70);
Query OK, 1 row affected (0.00 sec)

mysql> CALL heap_insert(203, 'Manoj', 90);
Query OK, 1 row affected (0.00 sec)

mysql> CALL heap_delete(202);
Query OK, 1 row affected (0.01 sec)

mysql> CALL heap_scan();
+-----+-----+-----+-----+-----+
| slot_no | regno | name   | marks | status |
+-----+-----+-----+-----+-----+
|      1 |    201 | Kavin  |    80 | ACTIVE |
|      2 |    202 | Lalitha |    70 | DELETED |
|      3 |    203 | Manoj  |    90 | ACTIVE |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> |

```

Question 2

Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

Note: Hash function $h(k) = \text{regno} \text{ MOD } 10$ (10 fixed buckets — this is what makes it 'static'). Collisions are handled by chaining: each bucket keeps a growing chain_pos sequence.

```
USE dbms_lab;

CREATE TABLE hash_table (
    bucket_no INT,
    chain_pos INT,
    regno INT,
    name VARCHAR(50),
    PRIMARY KEY (bucket_no, chain_pos)
);

DELIMITER //

CREATE PROCEDURE hash_insert(IN p_regno INT, IN p_name VARCHAR(50))
BEGIN
    DECLARE v_bucket INT;
    DECLARE v_pos INT;

    SET v_bucket = p_regno % 10;          -- static hash function h(k) = k mod 10
    SELECT IFNULL(MAX(chain_pos), 0) + 1 INTO v_pos
    FROM hash_table WHERE bucket_no = v_bucket;

    INSERT INTO hash_table(bucket_no, chain_pos, regno, name)
    VALUES (v_bucket, v_pos, p_regno, p_name);
END //

CREATE PROCEDURE hash_search(IN p_regno INT)
BEGIN
    DECLARE v_bucket INT;
    SET v_bucket = p_regno % 10;
    SELECT * FROM hash_table WHERE bucket_no = v_bucket AND regno = p_regno;
END //

-- populate 500 sample records to prove chaining works at scale
CREATE PROCEDURE hash_populate()
BEGIN
    DECLARE i INT DEFAULT 1;
    WHILE i <= 500 DO
        CALL hash_insert(1000 + i, CONCAT('Student', i));
        SET i = i + 1;
    END WHILE;
END //

DELIMITER ;

-- Test
CALL hash_populate();
SELECT bucket_no, COUNT(*) AS chain_length
FROM hash_table GROUP BY bucket_no ORDER BY bucket_no;
CALL hash_search(1250);
```

OUTPUT

```
mysql>
mysql> CREATE PROCEDURE hash_search(IN p_regno INT)
-> BEGIN
->     DECLARE v_bucket INT;
->     SET v_bucket = p_regno % 10;
->     SELECT * FROM hash_table WHERE bucket_no = v_bucket AND regno = p_regno;
-> END //
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- populate 500 sample records to prove chaining works at scale
mysql> CREATE PROCEDURE hash_populate()
-> BEGIN
->     DECLARE i INT DEFAULT 1;
->     WHILE i <= 500 DO
->         CALL hash_insert(1000 + i, CONCAT('Student', i));
->         SET i = i + 1;
->     END WHILE;
-> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> -- Test
mysql> CALL hash_populate();
Query OK, 1 row affected (0.95 sec)

mysql> SELECT bucket_no, COUNT(*) AS chain_length
-> FROM hash_table GROUP BY bucket_no ORDER BY bucket_no;
+-----+-----+
| bucket_no | chain_length |
+-----+-----+
| 0         | 50          |
| 1         | 50          |
| 2         | 50          |
| 3         | 50          |
| 4         | 50          |
| 5         | 50          |
| 6         | 50          |
| 7         | 50          |
| 8         | 50          |
| 9         | 50          |
+-----+-----+
10 rows in set (0.00 sec)

mysql> CALL hash_search(1250);
+-----+-----+-----+-----+
| bucket_no | chain_pos | regno | name       |
+-----+-----+-----+-----+
| 0         | 25        | 1250  | Student250 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> |
```

Question 3

Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

```
USE dbms_lab;

CREATE TABLE sorted_file (
    regno INT PRIMARY KEY,
    name VARCHAR(50),
    marks INT
);

INSERT INTO sorted_file
SELECT regno, name, marks FROM students ORDER BY regno;

DELIMITER //

CREATE PROCEDURE binary_search_regno(IN p_key INT)
BEGIN
    DECLARE v_low INT DEFAULT 1;
    DECLARE v_high INT;
    DECLARE v_mid INT;
    DECLARE v_regno INT;
    DECLARE v_found BOOLEAN DEFAULT FALSE;

    SELECT COUNT(*) INTO v_high FROM sorted_file;

    WHILE v_low <= v_high AND NOT v_found DO
        SET v_mid = FLOOR((v_low + v_high) / 2);
        SELECT regno INTO v_regno FROM sorted_file
        ORDER BY regno LIMIT 1 OFFSET v_mid - 1;

        IF v_regno = p_key THEN
            SET v_found = TRUE;
            SELECT * FROM sorted_file WHERE regno = v_regno;
        ELSEIF v_regno < p_key THEN
            SET v_low = v_mid + 1;
        ELSE
            SET v_high = v_mid - 1;
        END IF;
    END WHILE;

    IF NOT v_found THEN
        SELECT CONCAT('Record with regno ', p_key, ' not found') AS result;
    END IF;
END //

DELIMITER ;

-- Test
CALL binary_search_regno(106);
CALL binary_search_regno(999);
```

OUTPUT

```
mysql>
mysql> INSERT INTO sorted_file
-> SELECT regno, name, marks FROM students ORDER BY regno;
ERROR 1062 (23000): Duplicate entry '101' for key 'sorted_file.PRIMARY'
mysql>
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE binary_search_regno(IN p_key INT)
-> BEGIN
->     DECLARE v_low INT DEFAULT 1;
->     DECLARE v_high INT;
->     DECLARE v_mid INT;
->     DECLARE v_regno INT;
->     DECLARE v_found BOOLEAN DEFAULT FALSE;
->
->     SELECT COUNT(*) INTO v_high FROM sorted_file;
->
->     WHILE v_low <= v_high AND NOT v_found DO
->         SET v_mid = FLOOR((v_low + v_high) / 2);
->         SELECT regno INTO v_regno FROM sorted_file
->         ORDER BY regno LIMIT 1 OFFSET v_mid - 1;
->
->         IF v_regno = p_key THEN
->             SET v_found = TRUE;
->             SELECT * FROM sorted_file WHERE regno = v_regno;
->         ELSEIF v_regno < p_key THEN
->             SET v_low = v_mid + 1;
->         ELSE
->             SET v_high = v_mid - 1;
->         END IF;
->     END WHILE;
->
->     IF NOT v_found THEN
->         SELECT CONCAT('Record with regno ', p_key, ' not found') AS result;
->     END IF;
-> END //
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corre

        IF v_regno = p_key THEN
            SET v_found = TRUE;
        ' at line 14
mysql>
mysql> DELIMITER ;
mysql>
mysql> -- Test
mysql> CALL binary_search_regno(106);
ERROR 1305 (42000): PROCEDURE dbms_lab.binary_search_regno does not exist
mysql> CALL binary_search_regno(999);
ERROR 1305 (42000): PROCEDURE dbms_lab.binary_search_regno does not exist
mysql> |
```

Question 4

Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.

Note: Order 4 => max 3 keys / max 4 children per node. Tracing the standard B-Tree insertion algorithm for this exact key sequence produces one root split, giving the final tree Root=[10,20], Children=[5,6,7], [12,17], [30].

```
USE dbms_lab;

CREATE TABLE btree_nodes (
    node_id    INT PRIMARY KEY,
    parent_id  INT NULL,
    node_keys  VARCHAR(50),      -- comma separated sorted keys (max 3 for order 4)
    is_leaf    BOOLEAN DEFAULT TRUE
);

-- Final structure obtained after inserting 10,20,5,6,12,30,7,17 (traced by hand below)
INSERT INTO btree_nodes VALUES
(1, NULL, '10,20', FALSE),    -- root (split occurred when 5th key overflowed it)
(2, 1,   '5,6,7',  TRUE),
(3, 1,   '12,17',  TRUE),
(4, 1,   '30',     TRUE);

DELIMITER //

-- SEARCH: walk root -> child -> leaf comparing against node_keys
CREATE FUNCTION fn_find_child(p_parent INT, p_key INT) RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE v_k1 INT; DECLARE v_k2 INT; DECLARE v_child INT;
    SELECT
        CAST(SUBSTRING_INDEX(node_keys, ',', 1) AS UNSIGNED),
        CAST(SUBSTRING_INDEX(node_keys, ',', -1) AS UNSIGNED)
    INTO v_k1, v_k2
    FROM btree_nodes WHERE node_id = p_parent;

    IF p_key < v_k1 THEN
        SELECT node_id INTO v_child FROM btree_nodes
        WHERE parent_id = p_parent ORDER BY node_id LIMIT 1 OFFSET 0;
    ELSEIF p_key < v_k2 THEN
        SELECT node_id INTO v_child FROM btree_nodes
        WHERE parent_id = p_parent ORDER BY node_id LIMIT 1 OFFSET 1;
    ELSE
        SELECT node_id INTO v_child FROM btree_nodes
        WHERE parent_id = p_parent ORDER BY node_id LIMIT 1 OFFSET 2;
    END IF;
    RETURN v_child;
END //

CREATE PROCEDURE btree_search(IN p_key INT)
BEGIN
    DECLARE v_leaf INT;
    SET v_leaf = fn_find_child(1, p_key);
    IF FIND_IN_SET(p_key, (SELECT node_keys FROM btree_nodes WHERE node_id = 1)) > 0
       OR FIND_IN_SET(p_key, (SELECT node_keys FROM btree_nodes WHERE node_id = v_leaf)) > 0
    THEN
        SELECT CONCAT('Key ', p_key, ' FOUND') AS result;
    ELSE
        SELECT CONCAT('Key ', p_key, ' NOT FOUND') AS result;
    END IF;
END
```



```

        END IF;
    END //

-- DISPLAY: level order traversal using a recursive CTE
DELIMITER ;

WITH RECURSIVE tree AS (
    SELECT node_id, parent_id, node_keys, is_leaf, 0 AS level
    FROM btree_nodes WHERE parent_id IS NULL
    UNION ALL
    SELECT b.node_id, b.parent_id, b.node_keys, b.is_leaf, t.level + 1
    FROM btree_nodes b JOIN tree t ON b.parent_id = t.node_id
)
SELECT level, node_id, node_keys, is_leaf FROM tree ORDER BY level, node_id;

-- Test search
CALL btree_search(17);
CALL btree_search(15);

```

OUTPUT

```

mysql>
mysql> -- DISPLAY: level order traversal using a recursive CTE
mysql> DELIMITER ;
mysql>
mysql> WITH RECURSIVE tree AS (
->     SELECT node_id, parent_id, node_keys, is_leaf, 0 AS level
->     FROM btree_nodes WHERE parent_id IS NULL
->     UNION ALL
->     SELECT b.node_id, b.parent_id, b.node_keys, b.is_leaf, t.level + 1
->     FROM btree_nodes b JOIN tree t ON b.parent_id = t.node_id
-> )
-> SELECT level, node_id, node_keys, is_leaf FROM tree ORDER BY level, node_id
+-----+-----+-----+-----+
| level | node_id | node_keys | is_leaf |
+-----+-----+-----+-----+
| 0     | 1       | 10,20     | 0       |
| 1     | 2       | 5,6,7     | 1       |
| 1     | 3       | 12,17     | 1       |
| 1     | 4       | 30        | 1       |
+-----+-----+-----+-----+
4 rows in set (0.01 sec)

mysql>
mysql> -- Test search
mysql> CALL btree_search(17);
+-----+
| result |
+-----+
| Key 17 FOUND |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> CALL btree_search(15);
+-----+
| result |
+-----+
| Key 15 NOT FOUND |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> |

```

Question 5

Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

Note: Leaves hold every key and are chained via `next_leaf`, which is the feature that makes B+ Tree range scans fast (no need to revisit the index once the starting leaf is found).

```
USE dbms_lab;

CREATE TABLE bplus_index (          -- internal/root node: routing keys only
    node_id    INT PRIMARY KEY,
    node_keys  VARCHAR(50)          -- separator keys, e.g. '10,20'
);

CREATE TABLE bplus_leaves (
    leaf_id    INT PRIMARY KEY,
    next_leaf  INT NULL             -- linked-list pointer to the next leaf
);

CREATE TABLE bplus_leaf_keys (
    leaf_id    INT,
    key_val    INT
);

-- Constructed tree for keys 5,6,7,10,12,17,20,30
INSERT INTO bplus_index VALUES (1, '10,20');
INSERT INTO bplus_leaves VALUES (1, 2), (2, 3), (3, NULL);
INSERT INTO bplus_leaf_keys VALUES
(1,5),(1,6),(1,7),
(2,10),(2,12),(2,17),
(3,20),(3,30);

DELIMITER //

CREATE PROCEDURE bplus_traverse()
BEGIN
    DECLARE v_leaf INT DEFAULT 1;
    DROP TEMPORARY TABLE IF EXISTS traverse_result;
    CREATE TEMPORARY TABLE traverse_result (key_val INT);

    WHILE v_leaf IS NOT NULL DO
        INSERT INTO traverse_result SELECT key_val FROM bplus_leaf_keys WHERE leaf_id =
v_leaf;
        SELECT next_leaf INTO v_leaf FROM bplus_leaves WHERE leaf_id = v_leaf;
    END WHILE;

    SELECT * FROM traverse_result ORDER BY key_val;
END //

CREATE PROCEDURE bplus_range_query(IN p_low INT, IN p_high INT)
BEGIN
    DECLARE v_leaf INT;

    -- locate the first leaf whose smallest key >= lower bound
    SELECT leaf_id INTO v_leaf FROM bplus_leaf_keys
    WHERE key_val >= p_low ORDER BY key_val LIMIT 1;

    DROP TEMPORARY TABLE IF EXISTS range_result;
    CREATE TEMPORARY TABLE range_result (key_val INT);
```

```

WHILE v_leaf IS NOT NULL DO
    INSERT INTO range_result
    SELECT key_val FROM bplus_leaf_keys
    WHERE leaf_id = v_leaf AND key_val BETWEEN p_low AND p_high;

    IF (SELECT MAX(key_val) FROM bplus_leaf_keys WHERE leaf_id = v_leaf) > p_high THEN
        SET v_leaf = NULL;           -- gone past the range, stop following leaves
    ELSE
        SELECT next_leaf INTO v_leaf FROM bplus_leaves WHERE leaf_id = v_leaf;
    END IF;
END WHILE;

SELECT * FROM range_result ORDER BY key_val;
END //

DELIMITER ;

-- Test
CALL bplus_traverse();
CALL bplus_range_query(10, 20);

```

OUTPUT

```

mysql>
mysql> -- Test
mysql> CALL bplus_traverse();
+-----+
| key_val |
+-----+
|      5 |
|      6 |
|      7 |
|     10 |
|     12 |
|     17 |
|     20 |
|     30 |
+-----+
8 rows in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> CALL bplus_range_query(10, 20);
+-----+
| key_val |
+-----+
|     10 |
|     12 |
|     17 |
|     20 |
+-----+
4 rows in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

mysql> |

```

Question 6

Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

Note: Dense index = exactly one index entry per data record.

```
USE dbms_lab;

CREATE TABLE data_file (
    regno INT PRIMARY KEY,
    name VARCHAR(50),
    marks INT
);
INSERT INTO data_file SELECT regno, name, marks FROM students ORDER BY regno;

CREATE TABLE dense_index (
    index_key INT PRIMARY KEY,      -- one entry for every record (dense)
    block_pointer INT
);

INSERT INTO dense_index
SELECT regno, ROW_NUMBER() OVER (ORDER BY regno) FROM data_file;

DELIMITER //

CREATE PROCEDURE dense_index_search(IN p_key INT)
BEGIN
    DECLARE v_ptr INT;
    SELECT block_pointer INTO v_ptr FROM dense_index WHERE index_key = p_key;

    IF v_ptr IS NULL THEN
        SELECT 'Key not found in index' AS result;
    ELSE
        SELECT * FROM data_file ORDER BY regno LIMIT 1 OFFSET v_ptr - 1;
    END IF;
END //

DELIMITER ;

-- Display the full index table
SELECT * FROM dense_index ORDER BY index_key;

-- Test search
CALL dense_index_search(105);
```

OUTPUT

```
mysql> SELECT * FROM dense_index ORDER BY index_key;
```

index_key	block_pointer
101	1
102	2
103	3
104	4
105	5
106	6
107	7
108	8
109	9
110	10

```
10 rows in set (0.00 sec)
```

Question 7

Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

Note: Bucket capacity = 2. When a full bucket's local depth equals the global depth, the directory doubles before the bucket is split — this is the core extendible-hashing rule.

```
USE dbms_lab;

CREATE TABLE eh_config (global_depth INT, bucket_capacity INT);
INSERT INTO eh_config VALUES (1, 2);

CREATE TABLE eh_buckets (
    bucket_id INT PRIMARY KEY,
    local_depth INT,
    bucket_keys VARCHAR(100)
);
INSERT INTO eh_buckets VALUES (1, 1, ''), (2, 1, '');

CREATE TABLE eh_directory (
    dir_index INT PRIMARY KEY,
    bucket_id INT
);
INSERT INTO eh_directory VALUES (0, 1), (1, 2);

DELIMITER //

CREATE PROCEDURE eh_insert(IN p_key INT)
BEGIN
    DECLARE v_gd INT; DECLARE v_cap INT; DECLARE v_dir_idx INT;
    DECLARE v_bucket_id INT; DECLARE v_ld INT; DECLARE v_keys VARCHAR(100);
    DECLARE v_count INT; DECLARE v_new_bucket_id INT; DECLARE v_k INT; DECLARE v_bit INT;

    SELECT global_depth, bucket_capacity INTO v_gd, v_cap FROM eh_config;
    SET v_dir_idx = p_key MOD POWER(2, v_gd); -- last global_depth bits select the
    directory slot
```

```

SELECT bucket_id INTO v_bucket_id FROM eh_directory WHERE dir_index = v_dir_idx;
SELECT local_depth, bucket_keys INTO v_ld, v_keys FROM eh_buckets WHERE bucket_id =
v_bucket_id;
SET v_count = IF(v_keys = '', 0, LENGTH(v_keys) - LENGTH(REPLACE(v_keys, ',', '')) + 1);

IF v_count < v_cap THEN
    SET v_keys = IF(v_keys = '', CAST(p_key AS CHAR), CONCAT(v_keys, ',', p_key));
    UPDATE eh_buckets SET bucket_keys = v_keys WHERE bucket_id = v_bucket_id;
ELSE
    -- OVERFLOW: bucket is full
    IF v_ld = v_gd THEN
        -- local depth == global depth -> DOUBLE THE DIRECTORY
        INSERT INTO eh_directory (dir_index, bucket_id)
        SELECT dir_index + POWER(2, v_gd), bucket_id FROM eh_directory;
        UPDATE eh_config SET global_depth = v_gd + 1;
        SET v_gd = v_gd + 1;
    END IF;

    SELECT IFNULL(MAX(bucket_id), 0) + 1 INTO v_new_bucket_id FROM eh_buckets;
    INSERT INTO eh_buckets(bucket_id, local_depth, bucket_keys) VALUES (v_new_bucket_id,
v_ld + 1, '');
    UPDATE eh_buckets SET local_depth = v_ld + 1 WHERE bucket_id = v_bucket_id;

    SET v_keys = CONCAT(v_keys, ',', p_key);
    UPDATE eh_buckets SET bucket_keys = '' WHERE bucket_id IN (v_bucket_id,
v_new_bucket_id);

    WHILE LENGTH(v_keys) > 0 DO
        IF LOCATE(',', v_keys) > 0 THEN
            SET v_k = CAST(SUBSTRING_INDEX(v_keys, ',', 1) AS UNSIGNED);
            SET v_keys = SUBSTRING(v_keys, LOCATE(',', v_keys) + 1);
        ELSE
            SET v_k = CAST(v_keys AS UNSIGNED);
            SET v_keys = '';
        END IF;
        SET v_bit = FLOOR(v_k / POWER(2, v_ld)) MOD 2;    -- redistribute using the new
bit
        IF v_bit = 0 THEN
            UPDATE eh_buckets SET bucket_keys = IF(bucket_keys = '', CAST(v_k AS CHAR),
CONCAT(bucket_keys, ',', v_k)) WHERE bucket_id = v_bucket_id;
        ELSE
            UPDATE eh_buckets SET bucket_keys = IF(bucket_keys = '', CAST(v_k AS CHAR),
CONCAT(bucket_keys, ',', v_k)) WHERE bucket_id = v_new_bucket_id;
        END IF;
    END WHILE;

    UPDATE eh_directory
    SET bucket_id = v_new_bucket_id
    WHERE bucket_id = v_bucket_id
    AND FLOOR(dir_index / POWER(2, v_ld)) MOD 2 = 1;
END IF;
END //

DELIMITER ;

-- Insert 8 records to trigger directory doubling
CALL eh_insert(1); CALL eh_insert(7); CALL eh_insert(3); CALL eh_insert(12);
CALL eh_insert(20); CALL eh_insert(5); CALL eh_insert(9); CALL eh_insert(15);

SELECT * FROM eh_config;

```

```
SELECT * FROM eh_directory ORDER BY dir_index;
SELECT * FROM eh_buckets ORDER BY bucket_id;
```

OUTPUT

```
mysql>
mysql> SELECT * FROM eh_config;
+-----+-----+
| global_depth | bucket_capacity |
+-----+-----+
|             3 |                2 |
+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM eh_directory ORDER BY dir_index;
+-----+-----+
| dir_index | bucket_id |
+-----+-----+
|         0 |         1 |
|         1 |         2 |
|         2 |         1 |
|         3 |         3 |
|         4 |         1 |
|         5 |         4 |
|         6 |         1 |
|         7 |         5 |
+-----+-----+
8 rows in set (0.00 sec)

mysql> SELECT * FROM eh_buckets ORDER BY bucket_id;
+-----+-----+-----+
| bucket_id | local_depth | bucket_keys |
+-----+-----+-----+
|         1 |           1 | 12,20       |
|         2 |           3 | 1,9         |
|         3 |           3 | 3           |
|         4 |           3 | 5           |
|         5 |           3 | 7,15        |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

Question 8

Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

```
USE dbms_lab;

CREATE TABLE block_config (
    total_records      INT,
    records_per_block INT
);
INSERT INTO block_config VALUES (10000, 50);

DELIMITER //

CREATE PROCEDURE io_cost_analysis(IN p_search_key INT)
BEGIN
    DECLARE v_total INT; DECLARE v_per_block INT;
    DECLARE v_total_blocks INT; DECLARE v_seq_io INT; DECLARE v_indexed_io INT;

    SELECT total_records, records_per_block INTO v_total, v_per_block FROM block_config;
    SET v_total_blocks = CEIL(v_total / v_per_block);

    -- Sequential scan: worst case reads every block once
    SET v_seq_io = v_total_blocks;

    -- Indexed retrieval via a multi-level index of fan-out = records_per_block
    SET v_indexed_io = CEIL(LOG(v_total) / LOG(v_per_block)) + 1; -- index levels + 1 data
    block

    SELECT p_search_key AS search_key,
           v_total_blocks AS total_data_blocks,
           v_seq_io      AS sequential_scan_io,
           v_indexed_io  AS indexed_search_io;
END //

DELIMITER ;

-- Test
CALL io_cost_analysis(4521);
```

OUTPUT/

```
mysql>
mysql> DELIMITER ;
mysql>
mysql> -- Test
mysql> CALL io_cost_analysis(4521);
+-----+-----+-----+-----+
| search_key | total_data_blocks | sequential_scan_io | indexed_search_io |
+-----+-----+-----+-----+
|      4521 |             200 |             200 |          4 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```


Question 9

Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.

Note: Sparse index stores one entry per BLOCK (here, block size = 5 records) instead of one per record, trading a slightly slower in-block scan for a much smaller index.

```
USE dbms_lab;  -- uses data_file and dense_index created in Q6

CREATE TABLE sparse_index (
    index_key      INT PRIMARY KEY,
    block_pointer  INT
);

INSERT INTO sparse_index
SELECT regno, block_no FROM (
    SELECT regno,
           CEIL(ROW_NUMBER() OVER (ORDER BY regno) / 5) AS block_no,
           ROW_NUMBER() OVER (ORDER BY regno) AS rn
    FROM data_file
) t
WHERE rn % 5 = 1;      -- keep only the FIRST record of every 5-record block

DELIMITER //

CREATE PROCEDURE sparse_search(IN p_key INT)
BEGIN
    DECLARE v_block INT;
    SELECT block_pointer INTO v_block FROM sparse_index
    WHERE index_key <= p_key ORDER BY index_key DESC LIMIT 1;

    -- scan the 5 records of the located block for the exact key
    SELECT * FROM data_file ORDER BY regno LIMIT 5 OFFSET (v_block - 1) * 5;
END //

DELIMITER ;

-- Timing comparison: dense index vs sparse index
SET @start_dense = NOW(6);
CALL dense_index_search(108);
SET @end_dense = NOW(6);

SET @start_sparse = NOW(6);
CALL sparse_search(108);
SET @end_sparse = NOW(6);

SELECT TIMEDIFF(MICROSECOND, @start_dense, @end_dense) AS dense_time_microsec,
       TIMEDIFF(MICROSECOND, @start_sparse, @end_sparse) AS sparse_time_microsec;

SELECT (SELECT COUNT(*) FROM dense_index) AS dense_index_entries,
       (SELECT COUNT(*) FROM sparse_index) AS sparse_index_entries;
```

OUTPUT / SCREENSHOT (paste here):

```
mysql>
mysql> SELECT TIMESTAMPDIF(MICROSECOND, @start_dense, @end_dense) AS dense_time_microsec,
->          TIMESTAMPDIF(MICROSECOND, @start_sparse, @end_sparse) AS sparse_time_microsec;
+-----+-----+
| dense_time_microsec | sparse_time_microsec |
+-----+-----+
|          11825     |          2733       |
+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> SELECT (SELECT COUNT(*) FROM dense_index) AS dense_index_entries,
->          (SELECT COUNT(*) FROM sparse_index) AS sparse_index_entries;
+-----+-----+
| dense_index_entries | sparse_index_entries |
+-----+-----+
|          10        |          2           |
+-----+-----+
1 row in set (0.01 sec)
```

Question 10

Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.

Note: Reuses the B+ Tree (bplus_index / bplus_leaves / bplus_leaf_keys) and procedures built in Q5.

```
USE dbms_lab;

-- Construction: already done in Q5 (bplus_index, bplus_leaves, bplus_leaf_keys)
-- Traversal: uses bplus_traverse() from Q5
CALL bplus_traverse();

-- Range query for keys between 15 and 45
CALL bplus_range_query(15, 45);

-- Equivalent plain-SQL cross-check using the underlying data
SELECT key_val FROM bplus_leaf_keys
WHERE key_val BETWEEN 15 AND 45
ORDER BY key_val;
```

OUTPUT

```

mysql>
mysql> -- Range query for keys between 15 and 45
mysql> CALL bplus_range_query(15, 45);
+-----+
| key_val |
+-----+
|      17 |
|      20 |
|      30 |
+-----+
3 rows in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> -- Equivalent plain-SQL cross-check using the underlying data
mysql> SELECT key_val FROM bplus_leaf_keys
      -> WHERE key_val BETWEEN 15 AND 45
      -> ORDER BY key_val;
+-----+
| key_val |
+-----+
|      17 |
|      20 |
|      30 |
+-----+
3 rows in set (0.00 sec)

```